Module 2

Test Planning & Strategy in Depth (SV/UVM)

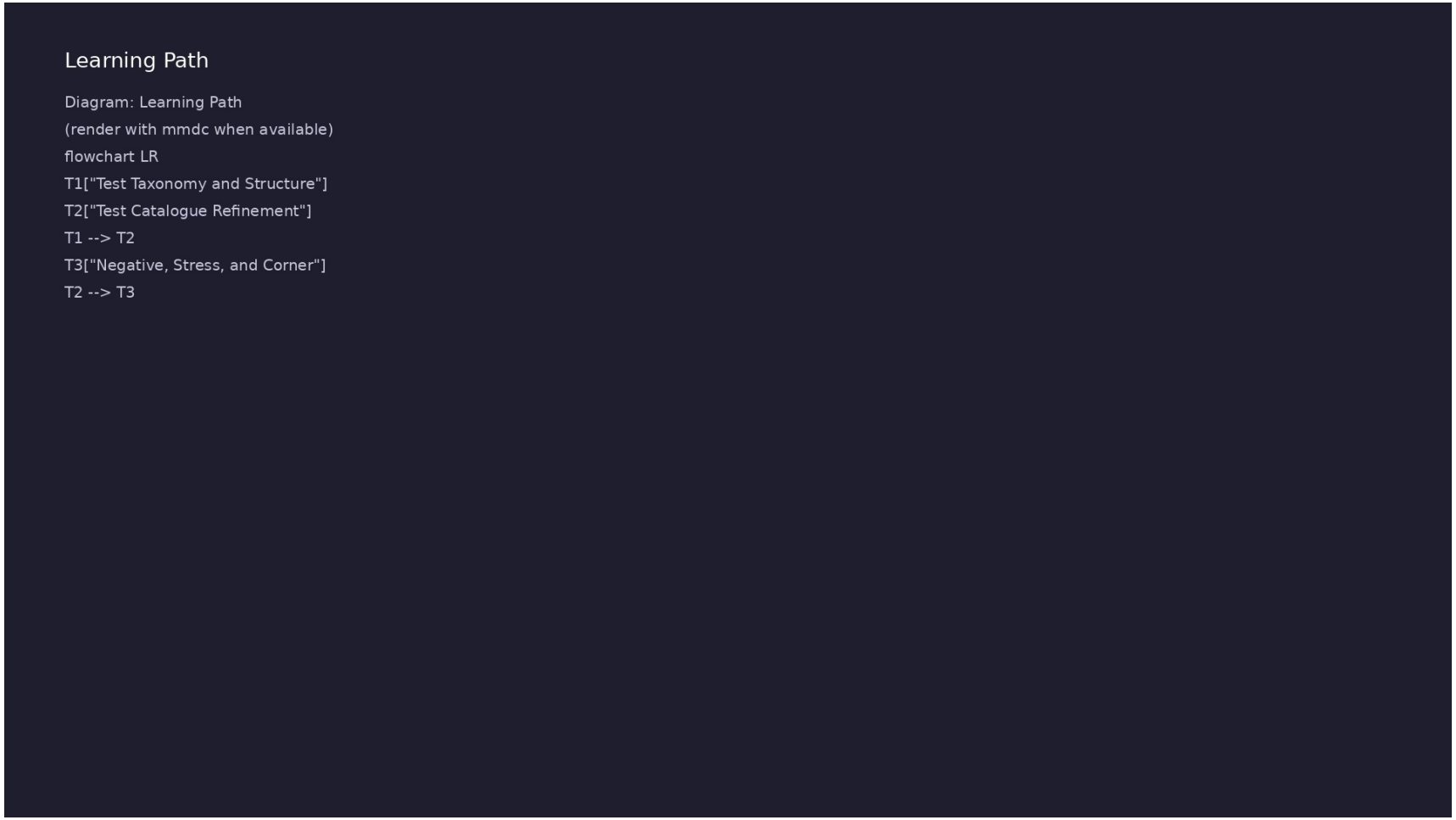
Learning objectives

- Turn your initial verification plan into a detailed, reviewable **test strategy**, with a structure…

Prerequisites

- See module README

Learning path



Turn your initial verification plan into a detailed, reviewable **test strategy**, with a structured test catalogue, negative/stress planni...

Overview

- Module 1 gave you a high-level verification plan, scope, and early test catalogue. In this module,...

Design architecture

1. UVM environment skeleton

- `common_dut/tb/stream_fifo_env_skeleton.sv` — env + agent stubs (source/sink agents).
- Hierarchy: `uvm_test` → `stream_fifo_env` → agents → (future) driver, sequencer, monitor.
- Interfaces tie to DUT ports via virtual interfaces or pin-level connections in the top testbench.

2. Test-plan ↔ TB mapping

- `TEST_PLAN.md` catalogue entries map to UVM tests, sequences, and config knobs.
- `REGRESSION_PLAN.md` defines tiers that will eventually select subsets of tests.
- Naming conventions (`SMK_`, `FTR_`, `ERR_`) link documentation to future `uvm_test` names.

Handshake logic (push/pop)

```
assign s_ready = (count < DEPTH);
  assign m_valid = (count != 0);
  assign level   = count;

  // Determine when to push/pop
  always_comb begin
    push = s_valid && s_ready;
    pop  = m_valid && m_ready;
  end
```

Agent stub — build_phase

```
class stream_fifo_agent_skeleton extends uvm_agent;

  `uvm_component_utils(stream_fifo_agent_skeleton)

  function new(string name = "stream_fifo_agent_skeleton",
uvm_component
parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // Stub: add driver, sequencer, monitor in later modules
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
```

Verification & testing methods

1. Test taxonomy

- Smoke/sanity — fast checks after every change; feature — per-requirement depth.
- Stress / long-run — backlog pressure, sustained traffic; error injection — overflow, underflow, ill...
- Each type documents expected runtime class and whether seeds are fixed or varied.

2. Regression tiering (planning level)

- Assign every catalogue test to sanity, core, stress, or full tiers.
- Document pass/fail policy preview and which negative tests are safe for frequent runs.
- `./scripts/module2.sh --check` verifies plan completeness and TB skeleton presence.

3. Reproducibility

- Plan seed strategy, constraint overrides, and plusargs per test for debuggable random runs.

Syllabus topics

1. Test Taxonomy and Structure (1/2)

- Test Types
- Smoke/sanity vs feature vs stress vs error-injection vs performance (if applicable).
- Directed vs constrained-random vs scenario-based tests.
- Tiers and Frequency
- Per-commit / pre-push tests.
- Nightly regression suites.

1. Test Taxonomy and Structure (2/2)

- Weekly full or long-running suites.
- Mapping to UVM
- How test types relate to UVM tests and sequences.
- Using configuration objects / plusargs / factory overrides to specialize tests.

2. Test Catalogue Refinement (1/2)

- From List to Catalogue
- Expanding the initial 10–20 test intents into a more complete catalogue.
- Grouping tests by feature, interface, or requirement group.
- Naming and Metadata
- Test ID conventions (e.g., `SMK_`, `FTR_`, `STR_`, `ERR_` prefixes).
- Metadata: owner, runtime class (short/medium/long), deterministic vs random, seed handling.

2. Test Catalogue Refinement (2/2)

- Redundancy and Value
- Identifying overlapping tests and merging where reasonable.
- Distinguishing “coverage-driving” tests vs “regression guardrail” tests.

3. Negative, Stress, and Corner-Case Planning (1/2)

- Negative Testing
- Illegal sequences, protocol violations, out-of-range values.
- Malformed packets/transactions, timing violations (where meaningful).
- Stress and Long-Run Scenarios
- High-load / long-duration tests.
- Pseudo-random traffic, back-to-back operations, resource exhaustion.

3. Negative, Stress, and Corner-Case Planning (2/2)

- Corner Cases
- Boundary values, wrap-around conditions, reset-in-the-middle, mode transitions.
- Instrumentation Needs
- What additional checks, monitors, or assertions are needed to support these tests.

4. Seeds, Constraints, and Reproducibility (1/2)

- Seed Strategy
- When to fix seeds vs vary them.
- How to record seeds for debugging and reproducibility.
- Constraints and Randomization Control
- High-level constraints in sequences vs constraint overrides per test.
- Guarding against “over-constrained” tests that hide bugs.

4. Seeds, Constraints, and Reproducibility (2/2)

- Test Parameters and Configurations
- Using UVM configuration database, plusargs, or config files.
- Planning for parametrization (e.g., data width, depth, mode).

5. Regression Tiering and Policies (Planning Level) (1/2)

- Regression Tiers
- Tier definitions (e.g., `sanity`, `core_feature`, `stress`, `full`).
- Target runtimes and approximate test counts for each tier.
- Test Assignment
- Mapping each test in the catalogue to one or more tiers.
- Deciding which negative/stress tests are safe for frequent runs.

5. Regression Tiering and Policies (Planning Level) (2/2)

- Pass/Fail Policies (Preview)
- What counts as a pass, fail, flake.
- Basic quarantine strategy for flaky tests (details in later modules).

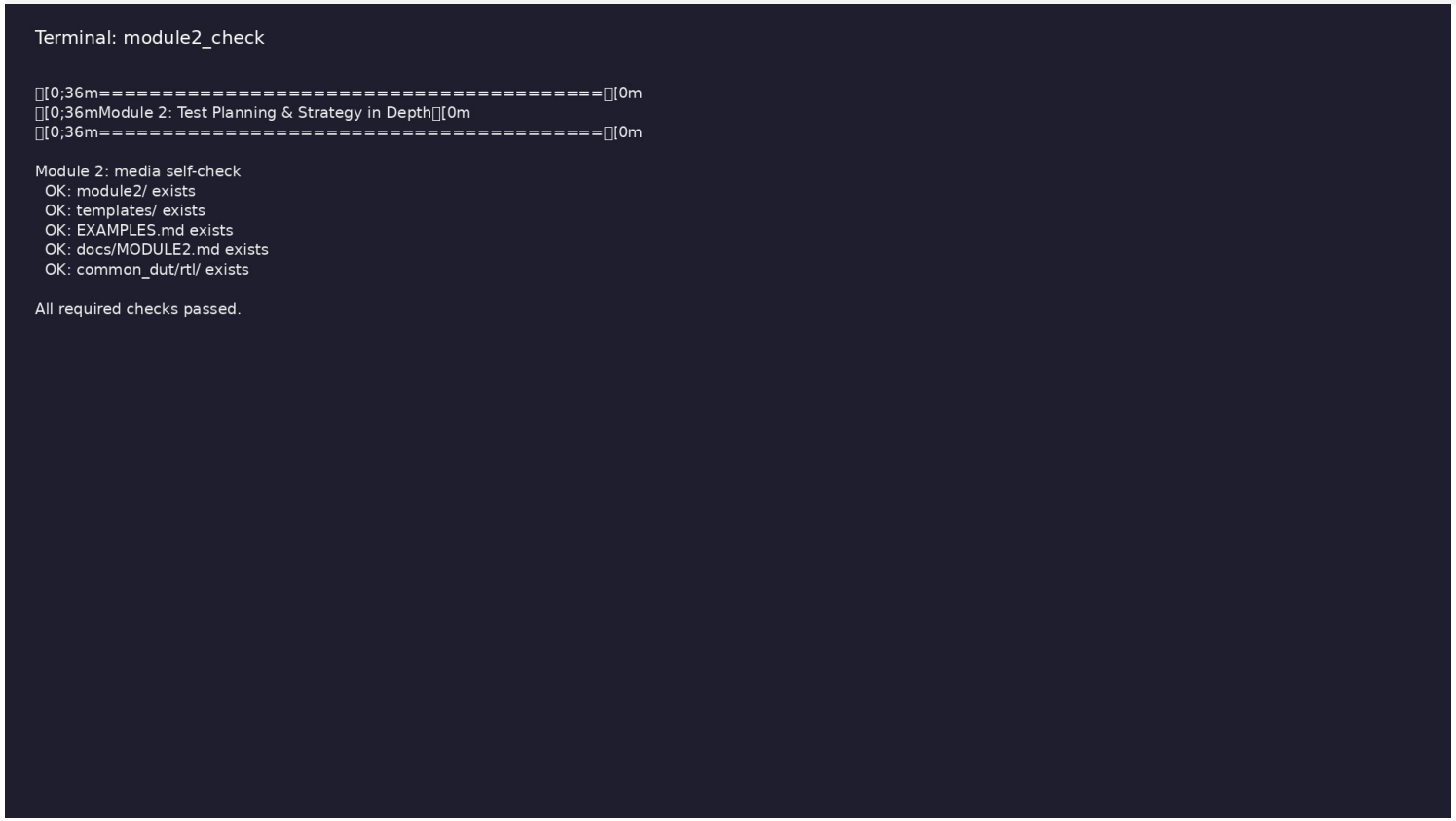
6. Coordination with Coverage and Requirements

- Closing Loops with Requirements
- Ensuring each high-priority requirement has enough tests (not just one).
- Verifying that tests exercise planned coverage points.
- Coverage-Driven Test Gaps
- Identifying missing tests from coverage perspective (e.g., unhit bins).
- Planning targeted tests to close meaningful coverage holes.

Hands-on examples

Module 2 self-check

```
$ ./scripts/module2.sh --test-plan
```



Exercise scaffold

```
./scripts/module2.sh --scaffold
```

Demo: Test plan template

```
$ head -30 module2/templates/TEST_PLAN.md
```

Terminal: ex01_test_plan

```
# Detailed Test Plan (Module 2)

> **Purpose**: Refine and expand the test catalogue and test strategy for your DUT.
> **Module**: 2 – Test Planning & Strategy in Depth (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module2/templates/` if nee

## 1. Context

<!-- TODO: Project name, DUT name, link to Module 1 artifacts. Summarize key decisions from Modu

## 2. Test Taxonomy

### 2.1 Test Types

<!-- TODO: Define test types (smoke, feature, stress, error, performance). -->

### 2.2 Test Tiers

<!-- TODO: Define tiers (e.g. SANITY, CORE, STRESS, FULL) with cadence and target runtime. -->

## 3. Test Metadata and Conventions

### 3.1 Naming and IDs

<!-- TODO: Test ID prefixes, UVM test class names, sequence naming. -->

### 3.2 Runtime and Stability

<!-- TODO: Runtime class, determinism, flake risk. -->
```

Demo: Regression plan template

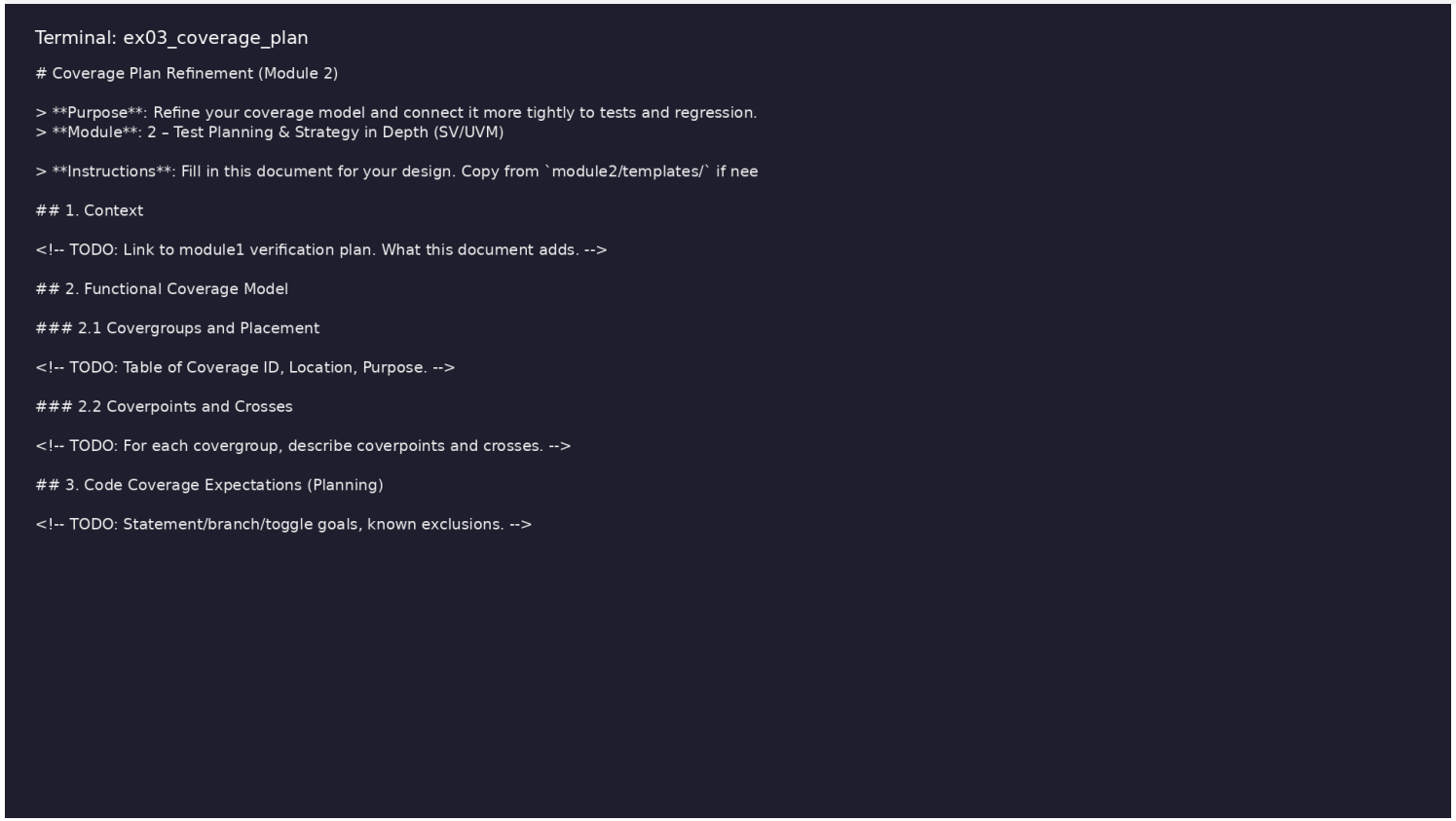
```
$ head -25 module2/templates/REGRESSION_PLAN.md
```

Terminal: ex02_regression_plan

```
# Regression Plan (Module 2)

> **Purpose**: Define initial regression tiers, policies, and test-to-tier mapping.
> **Module**: 2 – Test Planning & Strategy in Depth (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module2/templates/` if nee

## 1. Regression Goals

<!-- TODO: Primary goals (detect regressions, runtime targets, stability). -->

## 2. Regression Tiers

<!-- TODO: Table of Tier, Purpose, Contents, Cadence, Target Runtime. -->

## 3. Test Assignment to Tiers

<!-- TODO: Map test IDs from test_plan.md to tiers. -->

## 4. Runtime Budgeting

<!-- TODO: Per-tier runtime estimates and totals. -->

## 5. Pass/Fail, Flakes, and Quarantine (Planning-Level)
```

Demo: Coverage plan template

```
$ head -25 module2/templates/COVERAGE_PLAN.md
```

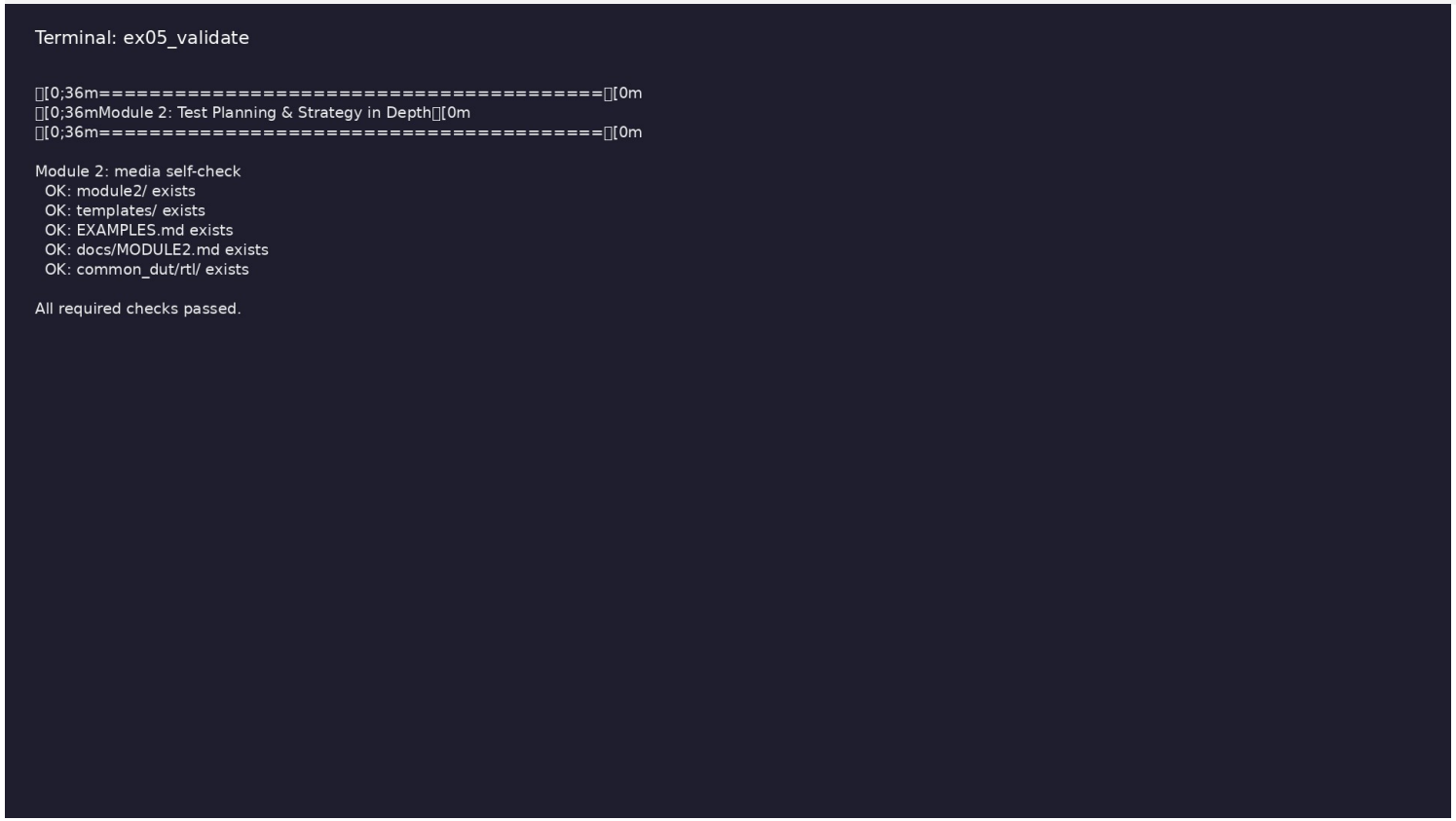


Demo: Reference test plan

```
$ head -20 module2/.solutions/TEST_PLAN.md
```

Demo: Module validation

```
$ ./scripts/module2.sh --check
```



Practice & assessment

What you should know (1/6)

- Design a structured test taxonomy tailored to your DUT and UVM environment.
- Maintain a rich test catalogue with clear metadata and priorities.
- Plan explicit negative, stress, and corner-case tests.
- Define initial regression tiers and assign tests to them.
- Explain how your test plan supports requirements coverage and coverage closure.
- In `module2/TEST_PLAN.md`, define:

What you should know (2/6)

- Test types with definitions and examples.
- Tiers (sanity, core, stress, full) with expected runtime budgets.
- For at least 10 existing tests from Module 1, assign:
- Type, tier, and expected runtime.
- Grow your catalogue to 25–40 tests (intents), focusing on:
- Feature coverage.

What you should know (3/6)

- Interface/protocol scenarios.
- Mode and configuration combinations.
- Identify at least 3 redundant or low-value tests
  and:
- Merge or remove them, documenting the decision.
- Add at least:
- 3 negative tests (illegal, malformed, or out-of-
  range behavior).

What you should know (4/6)

- 3 stress or long-run tests.
- For each, define:
- Preconditions and exact stimuli.
- Expected behavior (including assertions/flags).
- Special logging or debug support needed.
- Decide:

What you should know (5/6)

- Which tests will run with fixed vs varying seeds.
- How seeds will be passed (e.g., plusargs, config).
- Document this in `module2/TEST_PLAN.md` and update `module1/REQUIREMENTS_MATRIX.md` if needed.
- In `module2/REGRESSION_PLAN.md`, define:
- Tier names, cadence (per-commit, nightly, weekly), and runtime targets.
- Inclusion criteria for tests in each tier.

What you should know (6/6)

- For each test in your catalogue, add a column indicating:
- Primary tier(s) it belongs to.

Exercises

- Runtime Budgeting
- Flake-Resistance Design
- Traceability Check

Assessment checklist

- Has a written test taxonomy and tier definition.
- Test catalogue expanded and de-duplicated with clear metadata.
- Negative, stress, and corner-case tests are explicitly planned.
- Seed and randomness policy is documented.
- A draft regression plan exists with test-to-tier mapping.
- Requirements and coverage implications are reflected in the updated plan/matrix.

Summary & next steps

- Turn your initial verification plan into a detailed, reviewable **test strategy**, with a structure…
- Complete module2/CHECKLIST.md
- Review module2/EXAMPLES.md and run each lab
- Next: Coverage Planning & Analysis in Practice